REPUBLIQUE TUNISIENNE
****
Ministère de l'**E**nseignement **S**upérieur et
de la **R**echerche **S**cientifique
****
Faculté des **S**ciences de **T**unis



Support de cours :

Systèmes Logiques

Technologie de l'Informatique
Niveau 1

Elaboré par :

ZOUAOUI Ramzi

PH.D en Sciences de l'Informatique

Enseignant FST de Tunis

Année Universitaire 2015 – 2016

Préambule

Etablissement : Faculté des Sciences de Tunis

Discipline : Informatique

Niveau : 1ère semestre

Matière : Systèmes Logiques

Volume horaire : Cours 21H + T.D 10.5H + T.P 10.5H

Coefficient : 1.5

Crédit : 3

Régime : Mixte

Prérequis : Néant

Objectifs du cours :

Ce cours vise à faire acquérir à l'apprenant :

- les connaissances de base de l'électronique numérique.
- les capacités pour comprendre le fonctionnement des circuits logiques combinatoires et séquentiels qui sont à la base de l'architecture des ordinateurs et des systèmes numériques.

Moyens Pédagogiques :

- Support de cours
- Séries de travaux dirigés

Références Bibliographiques :

[1]. **Claude BRIE**, "Informatique Industrielle : Logique Combinatoire et Séquentielle : Méthode, Outil et Réalisation", 1ère édition, Technosup, Ellipse, 2003.

[2]. **Alexandre NKETSA**, "Informatique Industrielle : Circuit Logique Programmable : Mémoire, PLD, CPLD et FPGA", 1ère édition, Technosup, Ellipse, 1998.

Répartition de la matière :

	Titre	Objectifs Spécifiques :	Séances
Chap 1	Rappel et notions fondamentales pour l'électronique numérique	▪ Comprendre c'est quoi un système de numération. ▪ Apprendre les différents types de codage. ▪ Apprendre la méthode de conversion d'un système à un autre. ▪ Apprendre à faire des opérations arithmétiques en binaire.	**2**
Chap 2	Algèbre Booléenne & Portes Logiques	▪ La mise en équation d'un problème en algèbre de Boole. ▪ Maitriser la description d'une fonction par une table de vérité. ▪ Connaitre les propriétés des opérateurs logiques élémentaires et universels	**2**
Chap 3	Recherche et simplification des fonctions logiques	▪ Réaliser la synthèse d'une fonction logique à partir de circuits élémentaires. ▪ Réduire le nombre de termes et de variables dans une fonction. ▪ Apprendre des techniques simples pour simplifier une équation logique (Simplification algébrique, Diagramme de Karnaugh).	**2**
Chap 4	Etude de quelques circuits combinatoires (d'aiguillage, de transcodage, de comparaison)	▪ Connaître et comprendre : - La définition d'un circuit combinatoire. - Ses diverses représentations et implémentations. - Les circuits combinatoires usuels, leur fonctionnement et leurs applications. ▪ Être capable de : - Analyser un circuit combinatoire existant. - Réaliser un nouveau circuit combinatoire à partir d'une spécification.	**2**
Chap 5	Etude des circuits séquentiels : Les bascules (la bascule RS, RST, D, T, JK)	▪ Comprendre le fonctionnement des circuits séquentiels (asynchrone et synchrone). ▪ Découvrir les composants principaux de la logique séquentielle : les bascules. ▪ Maitriser pour chaque type de bascule : - Symbole. - Structure interne. - Le principe de fonctionnement. - Chronogramme des sorties.	**1**
Chap 6	Etude des circuits séquentiels : Les registres	▪ Définir les registres et leurs types. ▪ Maitriser la notion de décalage et de rotation. ▪ Etre capable de réaliser ces différents types de registres à partir des éléments de base (les bascules D et JK)	**1**
Chap 7	Etude des circuits séquentiels : Les compteurs	▪ Définir la notion de comptage/décomptage Synchrone et asynchrone. ▪ Réalisation des compteurs à partir des bascules JK ou bien des bascules T. ▪ Maitriser l'ajout des modulos de comptage.	**2**
Chap 8	Initiation aux circuits logiques programmables	▪ Comprendre les notions de base des circuits logiques programmables. ▪ Savoir l'avantage d'utilisation de ces circuits. ▪ Connaitre leurs types et leurs structures internes. ▪ Maitriser la réalisation des fonctions logique à travers ces types de circuits	**2**

Table des matières

Chapitre 1 : *Rappel et notions fondamentales pour l'électronique numérique* ***1***

1.1. Systèmes de numérotation *1*
1.2. Conversion entre les bases *2*
1.2.1. Base « B » vers la Base « 10 » *2*
1.2.2. Base « 10 » vers la Base « B » *2*
1.2.3. Base « 2n » vers la Base « 2 » *4*
1.2.4. Base « 2 » vers la Base « 2n » *5*
1.2.5. Base « i » vers la Base « j » *5*
1.3. Opérations dans les bases *5*
1.4. Codage des informations *6*
1.4.1. Le code binaire naturel et ses dérivés *6*
1.4.2. Le code DCB (Décimal Codé Binaire) *6*
1.4.3. Le code binaire réfléchi ou code de Gray *8*
1.4.4. Les codes alphanumériques *8*

Chapitre 2 : *Algèbre Booléenne & Portes Logiques* ***10***

2.1. Introduction *10*
2.2. Tables de vérité *11*
2.3. Les opérateurs logiques élémentaires *11*
2.3.1. L'opérateur « NON » : La complémentation *11*
2.3.2. L'opérateur « ET » : Produit logique *11*
2.3.3. L'opérateur « OU » : Addition logique *12*
2.4. Mise sous forme algébrique des circuits logiques *13*
2.5. Propriétés des opérateurs logiques : Théorèmes de Boole *13*
2.6. Théorème de Morgan *14*
2.7. Les opérateurs Universels *14*
2.7.1. L'opérateur « NON-ET » : NAND *14*
2.7.2. L'opérateur « NON-OU » : NOR *15*
2.7.3. L'opérateur « OU EXCLUSIF » : XOR *15*

Chapitre 3 : *Recherche et simplification des fonctions logiques* ***17***

3.1. Introduction *17*
3.2. Fonctions logiques décrites par une table de vérité *17*
3.2.1. Fonction complètement définie *17*
3.2.2. Fonction incomplètement définie *17*
3.2.3. Recherche d'une fonction logique à partir de sa table de vérité *18*
3.3. Simplification algébrique *19*
3.4. Diagrammes de KARNAUGH *20*
3.4.1. Définition *20*
3.4.2. Exemple de représentations *20*
3.4.3. Règle de simplification : groupement de cases *21*
3.4.4. Illustration *22*
3.4.5. Les valeurs indifférentes ou non spécifiées *22*
3.4.6. Exercice d'application *22*

Chapitre 4 : *Etude de quelques circuits combinatoires (d'aiguillage, de transcodage, de comparaison)* ***24***

4.1. Introduction *24*
4.2. Les circuits PORTES *24*
4.3. Les circuits combinatoires de transcodage *25*
4.3.1. Définition *25*
4.3.2. Les codeurs *25*
4.3.3. Les décodeurs *26*
4.3.4. Les transcodeurs *27*
4.4. Les circuits combinatoires de comparaison : les comparateurs *28*
4.5. Les circuits combinatoires d'aiguillage *29*

Chapitre 5 : *Etude des circuits séquentiels : Les bascules (la bascule RS, RST, D, T, JK)* ***32***

5.1. Les circuits séquentiels *32*
5.2. Les bascules : Définition *32*
5.3. Les bascules asynchrones : La bascule RS (Reset Set) *32*
5.4. Les bascules synchrones : *33*
5.4.1. La bascule RST *33*
5.4.2. La bascule D *34*
5.4.3. La bascule T *36*
5.4.4. La bascule JK *36*

Chapitre 6 : *Etude des circuits séquentiels : Les registres* ***38***

6.1. Définitions *38*
6.2. Registre à mémoire *38*
6.2.1. Exemple de réalisation à l'aide des bascules D *38*
6.2.2. Exemple de réalisation à l'aide des bascules JK *38*
6.3. Registre à décalage *39*
6.3.1. Types de décalages *39*
6.3.2. Types d'entrée-sortie *40*
6.3.3. Exemples de réalisation : registre à 4 bits *40*

Chapitre 7 : *Etude des circuits séquentiels : Les compteurs* ***43***

7.1. Définitions *43*
7.2. Compteurs Asynchrones (compteurs à propagation) *43*
7.2.1. Principe d'un compteur modulo 8 ..... *43*
7.2.2. Principe d'un décompteur modulo 8 ..... *44*
7.2.3. Conclusion *46*
7.2.4. Principe d'un compteur modulo X *46*
7.3. Compteurs Synchrones *47*

Chapitre 8 : *Initiation aux circuits logiques programmables* ***50***

8.1. Introduction *50*
8.2. Intérêts de la logique programmable *50*
8.3. Structure des réseaux logiques combinatoires *51*
8.4. Les composants logiques programmables (PLD) *53*
8.4.1. Les SPLD (Simple Programmable Logic Device) *54*
8.4.2. Les CPLD (Complex Programmable Logic Device) *56*

Chapitre 1 :

Rappel et notions fondamentales pour l'électronique numérique

1.1. Systèmes de numérotation

En technologie numérique, il existe plusieurs systèmes de numérotation, appelés « base ».

Les bases les plus utilisées sont :

• **Base Décimale :** 10 symboles : {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

Exemple :

$(2425)_{10} = 5.10^0 + 2.10^1 + 4.10^2 + 2.10^3$

$= 5 + 20 + 400 + 2000$

$= (2425)_{10}$

• **Base Binaire :** 2 symboles : {0, 1}

Exemple :

$(110100)_2 = 0.2^0 + 0.2^1 + 1.2^2 + 0.2^3 + 1.2^4 + 1.2^5$

$= 0 + 0 + 4 + 0 + 16 + 32$

$= (52)_{10}$

• **Base Octale :** 8 symboles : {0, 1, 2, 3, 4, 5, 6, 7}

Exemple :

$(2425)_8 = 5.8^0 + 2.8^1 + 4.8^2 + 2.8^3$

$= 5 + 16 + 256 + 1024$

$= (1301)_{10}$

• **Base Hexadécimale :** 16 symboles : {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F}

Exemple :

$(2425)_{16} = 5.16^0 + 2.16^1 + 4.16^2 + 2.16^3$

$= 5 + 32 + 1024 + 8192$

$= (9253)_{10}$

Règle Générale :

Dans le système de numérotation de base « B » le nombre $(N)_B$ s'écrit :

$(N)_B = (a_n\ a_{n-1} \ldots a_2\ a_1\ a_0)_B$

$= (a_n.B^n + a_{n-1}.B^{n-1} + \ldots a_2.B^2 + a_1.B^1 + a_0.B^0)_{10}$

L'équivalence entre les quatre représentations (Base 10, Base 2, Base 8, Base 16) est donnée par le tableau suivant :

Base 10	***Base 2***	***Base 8***	***Base 16***
0	0 0000	0	0
1	0 0001	1	1
2	0 0010	2	2
3	0 0011	3	3
4	0 0100	4	4
5	0 0101	5	5
6	0 0110	6	6
7	0 0111	7	7
8	0 1000	10	8
9	0 1001	11	9
10	0 1010	12	A
11	0 1011	13	B
12	0 1100	14	C
13	0 1101	15	D
14	0 1110	16	E
15	0 1111	17	F
16	1 0000	20	10
17	1 0001	21	11
18	1 0010	22	12
19	1 0011	23	13
20	1 0100	24	14

Remarque : Pour les nombres fractionnels, les puissances de la base deviennent négatives après la virgule.

En numérotation décimale, le nombre $(78,13)_{10}$ s'écrit :

$$(78,13)_{10} = \mathbf{7}.10^{1} + \mathbf{8}.10^{0} + \mathbf{1}.10^{-1} + \mathbf{3}.10^{-2}$$

En numérotation binaire, le nombre $(1011,101)_2$ s'écrit :

$$(1011,101)_2 = \mathbf{1}.2^{3} + \mathbf{0}.2^{2} + \mathbf{1}.2^{1} + \mathbf{1}.2^{0} + \mathbf{1}.2^{-1} + \mathbf{0}.2^{-2} + \mathbf{1}.2^{-3}$$

1.2. Conversion entre les bases

1.2.1. Base « B » vers la Base « 10 »

Pour convertir un nombre d'une base « B » vers la base « 10 » on utilise la règle générale.

Exemple :

$B = 2$: $N = (110010)_2 = 1.2^5 + 1.2^4 + 0.2^3 + 0.2^2 + 1.2^1 + 0.2^0 = (50)_{10}$

$B = 8$: $N = (1452)_8 = 1.8^3 + 4.8^2 + 5.8^1 + 2.8^0 = (810)_{10}$

$B = 16$: $N = (14AF)_{16} = 1.16^3 + 4.16^2 + 10.16^1 + 15.16^0 = (5295)_{10}$

1.2.2. Base « 10 » vers la Base « B »

1ère méthode :

Cette méthode se base sur la soustraction successive de la grande puissance. Cette méthode convient pour les nombres petits.

Exemple 1 : *(B=10) vers (B=2) :*

$(287)_{10} = (\ ?\ )_2$

$2^9=512 > 287 > 2^8=256$

$\Rightarrow (287)_{10} = a_8.2^8 + a_7.2^7 + a_6.2^6 + a_5.2^5 + a_4.2^4 + a_3.2^3 + a_2.2^2 + a_1.2^1 + a_0.2^0$

$\Rightarrow (287)_{10} = (a_8\, a_7\, a_6\, a_5\, a_4\, a_3\, a_2\, a_1\, a_0)_2$

287 – 256 = 31	$287 = 1.2^8 + 31$	$\Rightarrow a_8 = 1$
31 – 128 = nég	$31 = 0.2^7 + 31$	$\Rightarrow a_7 = 0$
31 – 64 = nég	$31 = 0.2^6 + 31$	$\Rightarrow a_6 = 0$
31 – 32 = nég	$31 = 0.2^5 + 31$	$\Rightarrow a_5 = 0$
31 – 16 = 15	$31 = 1.2^4 + 15$	$\Rightarrow a_4 = 1$
15 – 8 = 7	$15 = 1.2^3 + 7$	$\Rightarrow a_3 = 1$
7 – 4 = 3	$7 = 1.2^2 + 3$	$\Rightarrow a_2 = 1$
3 – 2 = 1	$3 = 1.2^1 + 1$	$\Rightarrow a_1 = 1$
1 – 1 = 0	$1 = 1.2^0 + 0$	$\Rightarrow a_0 = 1$

$\Rightarrow (287)_{10} = (100011111)_2$

Exemple 2 : *(B=10) vers (B=8) :*

$(287)_{10} = (\ ?\ )_8$

$8^3=512 > 287 > 8^2=64$

$\Rightarrow (287)_{10} = a_2.8^2 + a_1.8^1 + a_0.8^0$

$\Rightarrow (287)_{10} = (a_2\, a_1\, a_0)_8$

287 – 4x64 = 31	$287 = 4.8^2 + 31$	$\Rightarrow a_2 = 4$
31 – 3x8 = 7	$31 = 3.8^1 + 7$	$\Rightarrow a_1 = 3$
7 – 7x1 = 0	$7 = 7.8^0 + 0$	$\Rightarrow a_0 = 7$

$\Rightarrow (287)_{10} = (437)_8$

Exemple 3 : *(B=10) vers (B=16) :*

$(287)_{10} = (\ ?\ )_{16}$

$16^3=4096 > 287 > 16^2=256$

$\Rightarrow (287)_{10} = a_2.16^2 + a_1.16^1 + a_0.16^0$

$\Rightarrow (287)_{10} = (a_2\, a_1\, a_0)_{16}$

287 – 256 = 31	$287 = 1.16^2 + 31$	$\Rightarrow a_2 = 1$
31 – 16 = 15	$31 = 1.16^1 + 15$	$\Rightarrow a_1 = 1$
15 – 15x1 = 0	$7 = 15.16^0 + 0$	$\Rightarrow a_0 = 15$

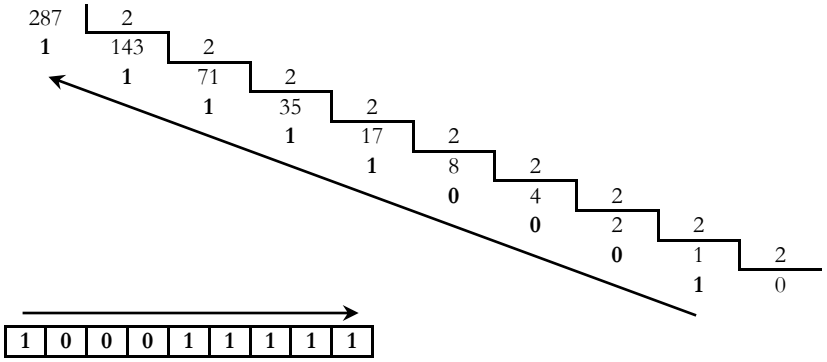
$\Rightarrow (287)_{10} = (11F)_8$

2ème méthode :

Cette méthode consiste **à diviser** par « B » autant de fois que cela est nécessaire pour obtenir **un quotient nul**, on écrit les restes de la division dans l'ordre inverse où ils sont obtenus.

Exemple 1 : *(B=10) vers (B=2) :*

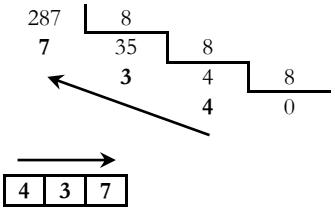
$(287)_{10} = (\ ?\ )_2$



$\Rightarrow (287)_{10} = (100011111)_2$

Exemple 2 : *(B=10) vers (B=8) :*

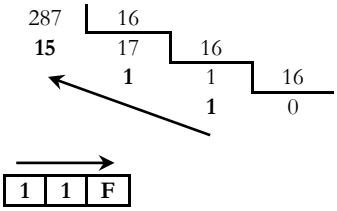
$(287)_{10} = (\ ?\ )_8$



$\Rightarrow (287)_{10} = (437)_8$

Exemple 3 : *(B=10) vers (B=16) :*

$(287)_{10} = (\ ?\ )_{16}$



$\Rightarrow (287)_{10} = (11F)_8$

1.2.3. Base « 2^n » vers la Base « 2 »

La conversion d'une base « 2^n » vers la base « 2 » se fait en développant chaque unité du nombre à convertir en un nombre binaire sur n bits.

Exemple 1 : *(B=8) vers (B=2) :*

$(347)_8 = (\ ?\ )_2$

$8 = 2^3 \Rightarrow$

	3			4			7	
0	1	1	1	0	0	1	1	1

$\Rightarrow (347)_8 = (011100111)_2$

Exemple 2 : *(B=16) vers (B=2) :*

$(A5C)_{16} = (\ ?\ )_2$

$16 = 2^4 \Rightarrow$

	A				5				C		
1	0	1	0	0	1	0	1	1	1	0	0

$\Rightarrow (A5C)_{16} = (101001011100)_2$

1.2.4. Base « 2 » vers la Base « 2^n »

La conversion d'une base « 2 » vers la base « 2^n » se fait en regroupant chaque n bits du nombre à convertir.

Exemple 1 : *(B=2) vers (B=8) :*

$(110110011100)_2 = (\ ?\ )_8$

$8 = 2^3 \Rightarrow$

1	1	0	1	1	0	0	1	1	1	0	0
	6			6			3			4	

$\Rightarrow (110110011100)_2 = (6634)_8$

Exemple 2 : *(B=2) vers (B=16) :*

$(110110011100)_2 = (\ ?\ )_{16}$

$16 = 2^4 \Rightarrow$

1	1	0	1	1	0	0	1	1	1	0	0
	D			9				C			

$\Rightarrow (110110011100)_2 = (D9C)_{16}$

1.2.5. Base « i » vers la Base « j »

- Si i et j sont tous les deux des puissances de 2, on utilise la base « 2 » comme base relais :

$$i \rightarrow 2 \rightarrow j$$

- Si i et j ne sont pas tous les deux des puissances de 2, on utilise la base « 10 » comme base relais :

$$i \rightarrow 10 \rightarrow j$$

1.3. Opérations dans les bases

Les opérations de base sont d'un nombre de quatre : l'addition, la soustraction, la multiplication et la division.

Les mêmes règles de calcul s'appliquent dans tous les systèmes de numérotation.

- En effectuant une addition, une retenue **(CARRY)** peut apparaître ; dans une soustraction il peut aussi y avoir une retenue **(BORROW)**. Enfin, si en manipulant des nombres trop grands pour le format, on obtient un dépassement de capacité **(OVERFLOW)** différent d'une retenue, ce dépassement doit être signalé par un drapeau **(FLAG)**.
- Les circuits numériques travaillent sur des nombres qui ont toujours la même longueur (FORMAT) ; par exemple, dans une machine de 8 bits le nombre 10111 doit être complété par des zéros 0001 0111

Addition	retenue	Soustraction	retenue	Multiplication	Division	Quotient	reste
0 + 0 = 0	0	0 - 0 = 0	0	0 x 0 = 0	0 : 0	Impossible	
0 + 1 = 1	0	0 - 1 = 1	1	0 x 1 = 0	0 : 1	0	0
1 + 0 = 1	0	1 - 0 = 1	0	1 x 0 = 0	1 : 0	Impossible	
1 + 1 = 0	1	1 - 1 = 0	0	1 x 1 = 1	1 : 1	1	0

- Il existe des circuits électroniques assez simples réalisant ces opérations de bases.
- Une soustraction se ramène à une addition à l'aide d'une représentation adéquate des nombres négatifs.
- Une multiplication s'effectue par les produits partiels (circuit ET), des décalages et des additions.
- La division se ramène à une suite de multiplication et de comparaison.

DEPASSEMENT DE CAPACITE :

Au niveau de l'ordinateur, on représente les nombres entiers positifs par un nombre fixe de bits (d'où l'appellation : représentation en champs fixe). Si N est le nombre de bits utilisés au niveau de l'ordinateur pour représenter les nombres alors seuls, les nombres Nbr tel que $0 \leq Nbr < 2N$ peuvent être représentés. Cela peut introduire des problèmes au niveau des opérations arithmétiques. En effet, une addition peut donner un résultat dépassant la valeur maximale possible.

Exemple :

Addition binaire sur 4 bits

```
        ¹1 1 1 1
+
        0 1 0 1
-------------------
=   1   0 1 0 0
```

Figure Erreur ! Il n'y a pas de texte répondant à ce style dans ce document.**–1** : *Exemple de dépassement suite à une addition*.

La valeur obtenue ne peut pas être représentée sur 4 bits, on parle **de débordement ou de dépassement de capacité (Overflow)**. Quand un débordement se produit, un indicateur est mis à 1. Dans certains ordinateurs, les calculs continuent, dans d'autres, le traitement s'arrête. Dans les deux cas, le problème est signalé.

1.4. Codage des informations

Un code est une correspondance entre un ensemble de symboles et un ensemble d'objets.

1.4.1. Le code binaire naturel et ses dérivés

Ce sont ceux que l'on utilise en arithmétique binaire : binaire, décimal, octal, hexadécimal.

1.4.2. Le code DCB (Décimal Codé Binaire)

Dans un nombre décimal, chaque chiffre (0, 1, 2, 3, 4, 5, 6, 7, 8, 9) est codé à l'aide de 4 bits (éléments binaires).

Exemple : $(153)_{10} = (0001\ 0101\ 0011)_{DCB} = (10011001)_2$

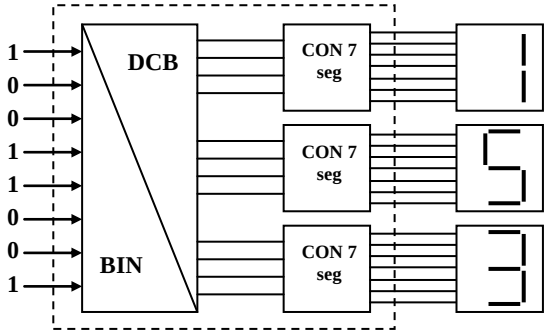
Ce code est pondéré avec les poids :

$a_0 = 1$, $a_1 = 2$, $a_2 = 4$, $a_3 = 8$, $a_5 = 10$, $a_6 = 20$, $a_7 = 40$, $a_8 = 80$, $a_9 = 100$, $a_{10} = 200$ …

Les groupes non valides dans ce code sont : 1010, 1011, 1100, 1101, 1110 et 1111.

Utilisation :

Pour afficher le nombre $(153)_{10}$ c'est-à-dire $(10011001)_2$ il suffit de le convertir en code DCB et de relier les signaux obtenus aux circuits afficheur ; le circuit de conversion s'appelle un transcodeur.



Travaille sur des nombres purs (représentés en octal ou en hexadécimal) est intéressant dans les calculateurs, mais lorsqu'on veut une image rapide de l'équivalent décimal, on est amené à effectuer un transcodage long et fastidieux.

Le code BCD consiste à remplacer chaque chiffre décimal du nombre par sa codification sur 4 bits en binaire

Exemple

```
  1    9      7    5       en décimal

0001 1001 | 0111 0101      en BCD.
```

Le code BCD permet une implémentation des additions simple à l'aide de l'addition duale présentée ci-dessous : les bits de même Quartet sont additionnés dans un premier temps comme s'il s'agissait d'un nombre binaire. Ensuite, on contrôle les valeurs des Quartets et on ajoute la valeur 6 chaque fois que la valeur d'un Quartet dépasse 9.

Exemple

```
      125
+
      628
 =    753
```

```
   0001  0010  0101
+
   0110  0010  1000
= 0111  0100  1101        > 1001 il faut
+                         ajouter 0110
   0000  0000  0110
= 0111  0101  0011
```

1.4.3. Le code binaire réfléchi ou code de Gray

Le code de Gray est un code non pondéré. Il appartient à la catégorie des codes dit à distance minimale, du fait qu'une représentation codée ne diffère de celle qui la précède que par un bit.

On dit que ces termes sont adjacents ne sont pas nécessairement côte à côte mais sont toujours symétriques par rapport à un axe.

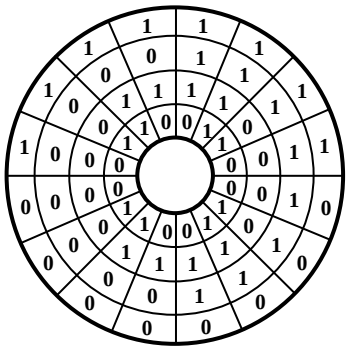
Binaire naturel	*Nombre Décimal*	*Binaire Réfléchi*	
0000	0	0000	
0001	1	0001	1er axe
0010	2	0011	
0011	3	0010	2ème axe
0100	4	0110	
0101	5	0111	
0110	6	0101	
0111	7	0100	3ème axe
1000	8	1100	
1001	9	1101	
1010	10	1111	
1011	11	1110	2ème axe
1100	12	1010	
1101	13	1011	1er axe
1110	14	1001	
1111	15	1000	

Utilisation :

Le code de Gray sert souvent dans des situations où d'autres codes, comme le code binaire, peuvent produire des résultats ambigus ou erronés au moment de transitions entraînant le changement de plusieurs bits dans le code.

Par exemple, dans le code binaire, lorsqu'on passe de 0111 à 1000 ; les quatre bits changent en même temps (la transition peut occasionner un ou plusieurs états intermédiaires dans les circuits).

Ce code permet de coder des positions angulaires sans discontinuité et sans état de parasite entre deux états successifs.



1.4.4. Les codes alphanumériques

Le code alphanumérique le plus répondu est le code ASCII (American Standard Code for Information Interchange) qui est codé 7 bits. Grâce à ce code, on peut représenter $2^7 = 128$ groupe codés. Le tableau suivant contient la liste de ce code.

Ce code est utilisé par la majorité des micro-ordinateurs. Il sert aussi à coder l'information alphanumérique transmise entre un ordinateur et ses périphériques d'entrées-sorties. Dans ce type d'utilisation, un huitième bit (bit de parité) peut être ajouté pour contrôler les erreurs de transmission.

Un huitième bit de parité est souvent adjoint de façon à faciliter la détection d'erreur.

				b_6	0	0	0	0	1	1	1	1
				b_5	0	0	1	1	0	0	1	1
				b_4	0	1	0	1	0	1	0	1
b_3	b_2	b_1	b_0		0	1	2	3	4	5	6	7
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	A	LF	SUB	*	:	J	Z	j	z
1	0	1	1	B	VT	ESC	+	;	K	[	k	{
1	1	0	0	C	FF	IS4	,	<	L	\	l	|
1	1	0	1	D	CR	IS3	-	=	M	]	m	}
1	1	1	0	E	SO	IS2	.	>	N	^	n	~
1	1	1	1	F	SI	IS1	/	?	O	_	o	DEL